

Machine Learning

Decision Trees

Random Forests

Donny Hurley

ATU

All classification models implement decision rules.

A decision tree makes these rules explicit and interpretable.

Instead of us writing conditions, the model learns them from data.

ML as Learned If / Else Rules

```
if ???:  
    return 'red'  
elif ???:  
    return 'orange'
```

For example, in digit recognition:

```
if img[4,8] < 30 and img[32,18] > 10:  
    return '1'
```

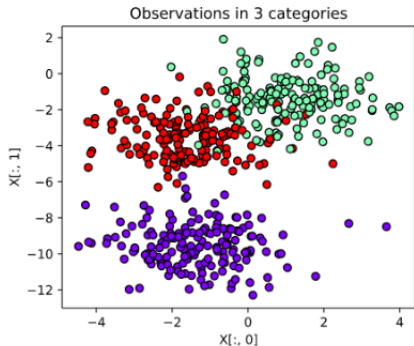
Decision trees learn these rules automatically.

Decision Trees: Core Idea

- Start with all training data.
- Choose a feature and a threshold.
- Split the data into two groups.
- Repeat recursively on each branch.
- Stop when further splitting is unnecessary.

The result is a tree of decisions.

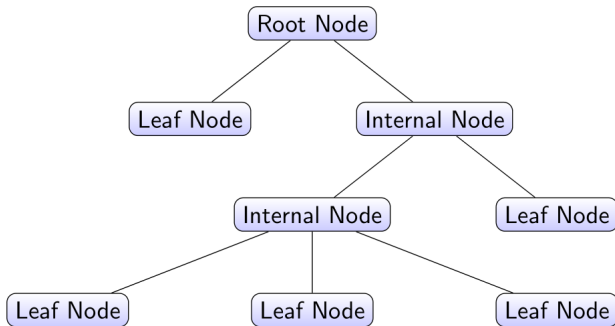
Example



Simple rules:

- If x is small, predict purple.
- Else if y is small, predict red.
- Else predict green.

Tree Terminology



- Root node: first split
- Internal nodes: decision points
- Leaf nodes: final predictions

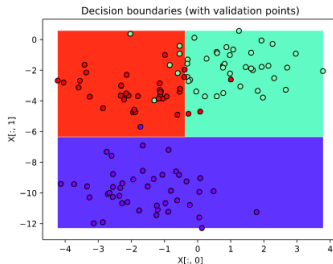
- Decision trees are a type of classifier network.
- They are very easy to interpret which means that if causal reasoning is important then they can be one of the better ways to go.
- This works because it is very easy to trace our way back up the tree to determine why each decision was made.
- In general, trees are upside down with the root at the top and the leaves at the bottom.
- Nodes that are neither the root or a leaf are called internal nodes. If we get to a leaf node, a classification has been made.

Decision Trees I

We can use scikit-learn to do the work

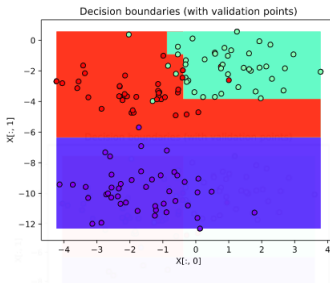
```
from sklearn.tree import DecisionTreeClassifier  
model = DecisionTreeClassifier(max_depth=2)  
model.fit(X_train, y_train)
```

Giving these decision boundaries



Decision Trees II

You'll notice `max_depth` is a hyperparameter. Increasing that will make things more complex, with the usual risk of overfitting. With this "toy" example and depth four we get the following decision boundaries



Another option: how many nodes must be on a branch before it's too small to split?

```
model = DecisionTreeClassifier(min_samples_split=30)
```

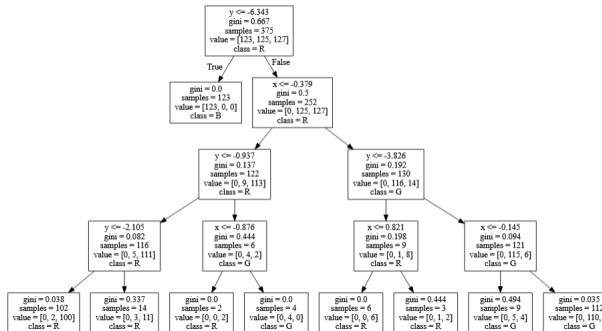
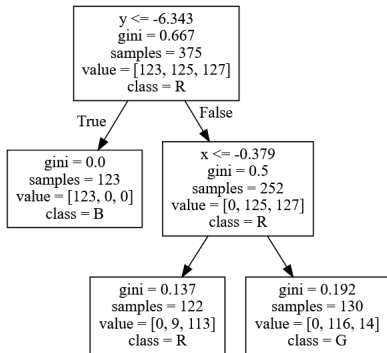
Important hyperparameters:

- `max_depth` – how many decisions are chained
- `min_samples_leaf` – minimum samples at a leaf
- `min_samples_split` - minimum number of samples required to split an internal node

You can of course combine them and set multiple hyperparameters.

- Deep trees: low bias, high variance
- Shallow trees: higher bias, lower variance

Scikit-learn can visualise the decision trees for us https://scikit-learn.org/stable/modules/generated/sklearn.tree.export_graphviz.html exporting to graphviz. For the two max depths we get



How Does a Tree Decide Where to Split?

At each node, the algorithm chooses the split that produces the **purest** child nodes. Purity measures how mixed the class labels are after the split.

Common impurity measures:

- Gini impurity (default in scikit-learn)
- Entropy
- Both are a measure of how "pure" a branch is. If we can split so that 100% of values on each side are in one category, then that's good. If we split so that every outcome is equally likely on each side, that's bad. Create the decision that maximizes "goodness".
- Either method seems to produce similar trees. In either case, we create a decision, and then recurse until we have to stop.

Gini impurity measures how mixed the classes are at a node.

$$G = 1 - \sum_{i=1}^c p_i^2$$

where p_i is the proportion of class i at the node.

- $G = 0 \Rightarrow$ node is perfectly pure
- Equal class proportions $\Rightarrow$ high Gini

When viewing a tree (e.g. Graphviz output):

- Gini is displayed at each node
- Leaf nodes with only one class have $Gini = 0$
- Internal nodes with mixed labels have $G > 0$

Tree examples

The default for `sklearn.tree.DecisionTreeClassifier` criterion is 'gini' but you can set it to 'entropy' like the following examples.

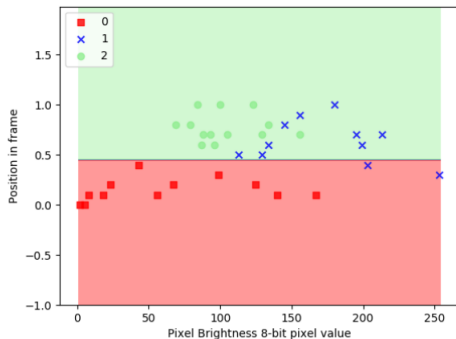


Figure: Decision Boundaries depth 1

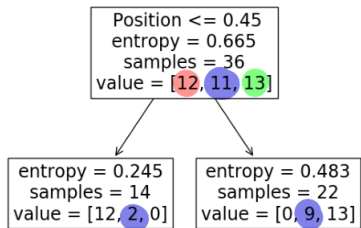


Figure: Tree for depth 1

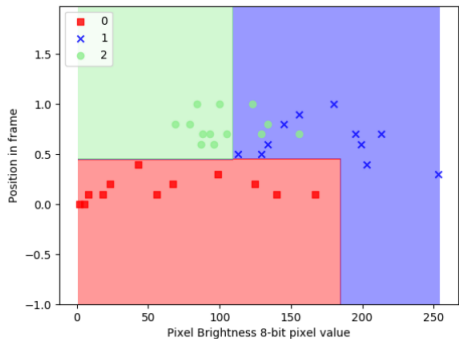


Figure: Decision Boundaries depth 2

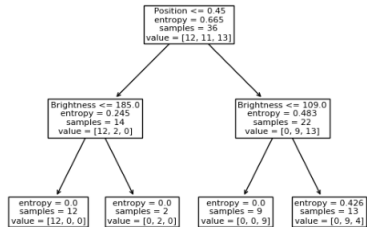


Figure: Tree for depth 2

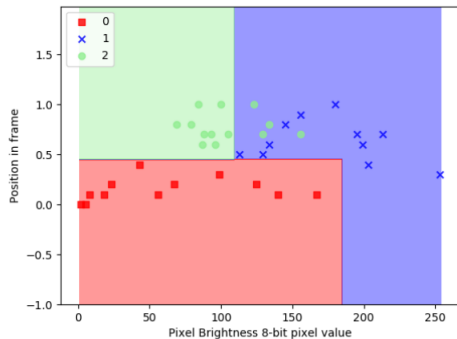


Figure: Decision Boundaries depth 3

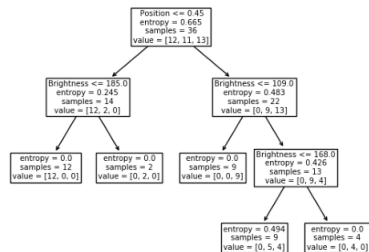


Figure: Tree for depth 3

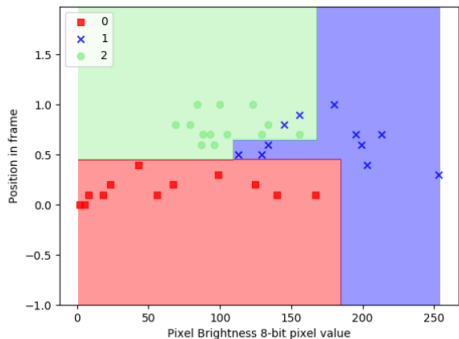


Figure: Decision Boundaries depth 4

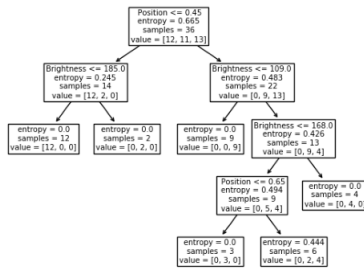


Figure: Tree for depth 4

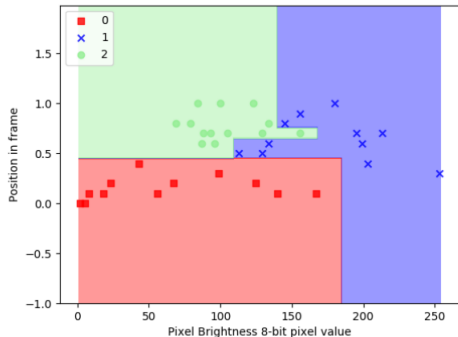


Figure: Decision Boundaries depth 6

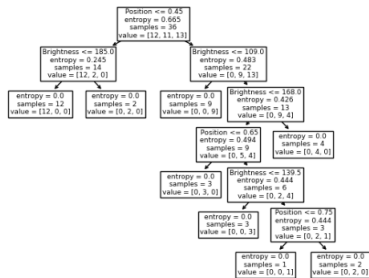


Figure: Tree for depth 6

Tree depth 1 Training Score 0.694444 Validation Score: 0.562500

Tree depth 2 Training Score 0.888889 Validation Score: 0.937500

Tree depth 3 Training Score 0.888889 Validation Score: 0.937500

Tree depth 4 Training Score 0.944444 Validation Score: 1.000000

Tree depth 5 Training Score 0.972222 Validation Score: 0.937500

Tree depth 6 Training Score 1.000000 Validation Score: 0.937500

Keep in mind that there are only 32 points in our training set and 16 points in our validation set so more data would certainly help the situation.

Training vs Validation Performance

As tree depth increases:

- Training accuracy increases
- Validation accuracy may decrease

This is overfitting.

Advantages of Decision Trees

- Highly interpretable
- Minimal preprocessing required
- Handle numeric and categorical data
- Capture nonlinear interactions
- Useful when explanation matters

Disadvantages of Decision Trees

- High variance
- Sensitive to small data changes
- Single trees rarely generalise optimally

- It can sometimes be hard to balance the complexity of the classifier with over-fitting. Sometimes, we'll find that no single classifier will do a great job on the problem.
- It can be easy to produce a classifier that makes predictions better than chance, but still not great.
- Often useful to have many simple classifiers, with some differences in their construction so they produce different results and let them vote: an ensemble
- An ensemble can often produce good results, even though the individual classifiers are quite bad.
- Combining many weak models can give a much stronger and robust result that generalises better.
- The wisdom of the crowd

Usually, all of the models will be of the same type, but they don't have to be. A VotingClassifier will let you combine any collection of models you want, train them together, and vote on predictions.

```
model = VotingClassifier([
    ('nb', GaussianNB()),
    ('knn', KNeighborsClassifier(5)),
    ('svm', SVC(kernel='linear', C=0.1)),
    ('tree1', DecisionTreeClassifier(max_depth=4)),
    ('tree2', DecisionTreeClassifier(min_samples_leaf=10)),
])
model.fit(X_train, y_train)
print(model.score(X_train, y_train))
print(model.score(X_valid, y_valid))
```

A random forest is an ensemble of decision trees where:

- Each tree is trained on a bootstrap sample
- Each split considers only a random subset of features
- Predictions are aggregated by voting

Random Forest in scikit-learn

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(
    n_estimators=100,
    max_depth=5,
    min_samples_leaf=10
)
model.fit(X_train, y_train)
```

Each decision tree will be constructed as before, but...

- individual trees will likely be simpler (lower max depth, fewer leaves, etc);
- when training, use only a sample of the training data (chosen randomly with replacement, a bootstrap sample);
- for each decision, consider only a random subset of the features.

... and repeat many times.

Random Forest 10 times

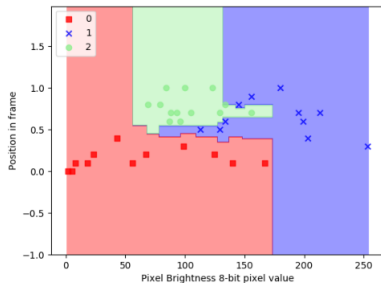


Figure: Random Forest 10 Decision
Trees with Gini
Gini Training Score 0.972222
Validation Score: 0.937500

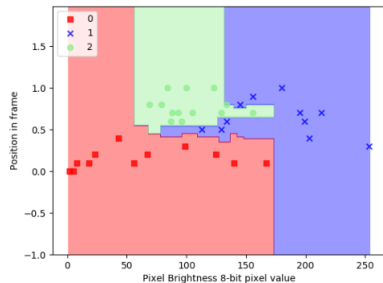


Figure: Random Forest 10 Decision
Trees with Entropy
Entropy Training Score 0.972222
Validation Score: 0.937500

Hyperparameters That Matter

Focus on:

- `max_depth`
- `min_samples_leaf`
- `n_estimators`

More trees $\Rightarrow$ lower variance but higher computation cost.

Random Forests I

Original toy data. When constructing a random forest, we get the same options as a decision tree about the tree shape, plus the number of estimators (and some other choices about the randomness of the construction).

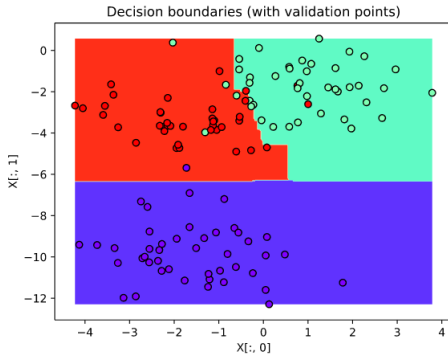
```
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100,
                              max_depth=3, min_samples_leaf=10)
model.fit(X_train, y_train)
print(model.score(X_train, y_train))
print(model.score(X_valid, y_valid))
```

```
0.96533333333333334
```

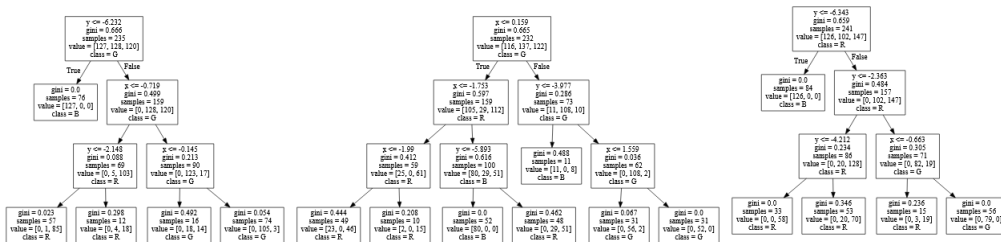
```
0.92
```

Random Forests II

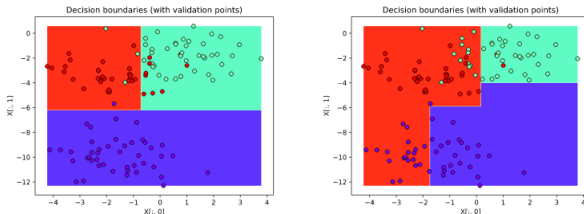
We get a reasonable-looking decision boundary



A few of the decision trees from the forest:



None of the individual trees make great decisions, but together they do.



The voting among trees can also help with over-fitting: since each was trained with a subset of the data, it's less of a problem.

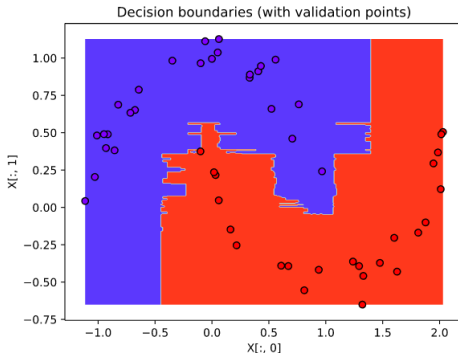
Let's try some data from `make_moons`. We can say very little about tree size but still get good results:

```
model = RandomForestClassifier(n_estimators=400, max_depth=7)
model.fit(X_train, y_train)
print(model.score(X_train, y_train))
print(model.score(X_valid, y_valid))

1.0
1.0
```

Random Forests VI

There is still maybe some over-fitting, but it's not affecting the scores.



Random Forest Algorithm

- ① Draw a random bootstrap sample of size n (randomly choose n samples from the training set with replacement i.e. there can be duplicates).
- ② Grow a decision tree from the bootstrap sample. At each node:
 - Randomly select d features without replacement i.e. no duplicates.
 - Split the node using the feature that provides the best split according to the objective function, for instance, by maximising the information gain.
- ③ Repeat the steps 1 and 2 k times.
- ④ Aggregate the prediction by each tree to assign the class label by majority vote.

Bias–Variance Summary

- Deep single tree: low bias, high variance
- Shallow tree: higher bias, lower variance
- Random forest: low variance, good generalisation

When to Use Trees and Random Forests

They work particularly well for:

- Tabular data
- Mixed feature types
- Nonlinear relationships
- Small to medium datasets
- Messy or noisy data

Trees vs Neural Networks

- Trees: tabular, interpretable, low preprocessing
- Neural networks: images, text, audio, large datasets
- Trees are often strong baselines
- Neural networks are rarely the best choice for tabular data

- Each tree leaves out $\approx 1/3$ of training samples
- These are called out-of-bag samples
- Can estimate generalisation internally
- Useful, but not a replacement for validation

Reproducibility with Random Forests and Cross Validation

Random forests contain randomness:

- Bootstrap sampling of training data
- Random subset of features at each split

To ensure consistent cross-validation results, fix the random seed:

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score

model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

%scores = cross_val_score(model, X_train, y_train)
```

Setting `random_state` ensures reproducible results across runs.

- Decision trees are interpretable but unstable
- Random forests reduce variance via averaging
- Excellent choice for tabular data
- Strong baselines before neural networks

The last two model types we studied were kNN and RandomForest. These both naturally handle multiple classes.

Logistic Regression and Support Vector Machines are *binary classifiers* at their core.

They are designed to separate two classes.

But in the lab we used LogisticRegression to classify multiple things.

How is that possible if it is a binary model?

We need strategies to extend binary classifiers to multi-class problems.

Three Ways to Extend Binary Classifiers

There are three main strategies:

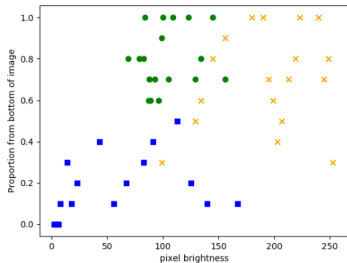
- One-vs-Rest (OvR)
- One-vs-One (OvO)
- Direct Multinomial Model (Softmax)

OvR and OvO are reduction strategies:

- Reduce a multi-class problem to many binary problems

Softmax is a direct multi-class model.

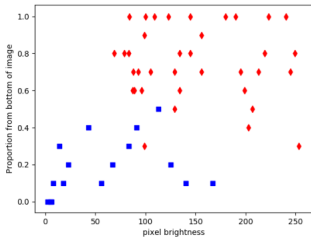
One-vs-Rest (OvR) (Sometimes called One-vs-all)



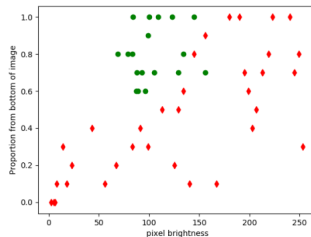
Three classes: Road, Road Sign, Other.

- Turn the k -class problem into k binary problems.
- Each classifier learns:

Class i vs All Other Classes
- This produces k models.



Road Vs All



Road Signs Vs All

To classify a new sample:

- Run it through all k classifiers.
- Each classifier produces a score or probability.
- Choose the class with the highest score.

OvR is now the default strategy for many SVM implementations.

Properties of One-vs-Rest

- Requires k classifiers.
- Simple and widely used.
- Can suffer from class imbalance:
 - One positive class
 - Many negatives
- Works well in practice.

Modern SVM implementations typically use OvR by default.

Another strategy is One-vs-One.

- Train a classifier for every pair of classes.
- Number of classifiers:

$$\frac{k(k-1)}{2}$$

- Final prediction is made by majority vote.

One-vs-One Example

For three classes A, B, C:

We train:

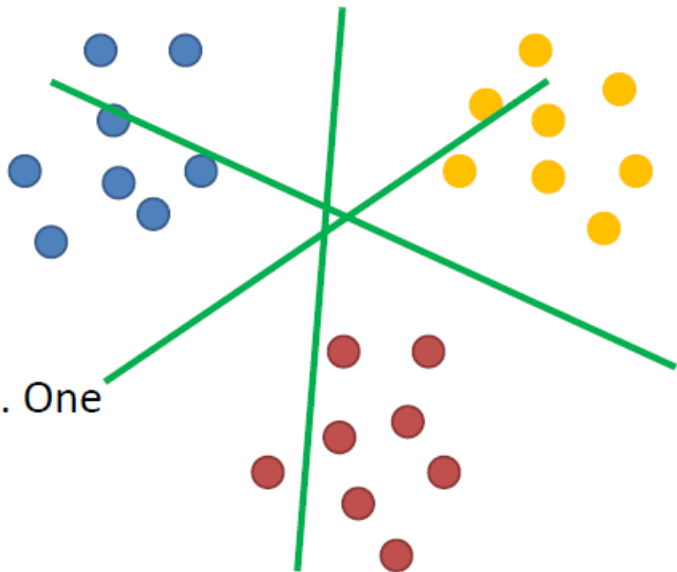
- A vs B
- A vs C
- B vs C

A new sample might produce:

(B, A, B)

Majority vote $\Rightarrow$ class B.

One vs. One



Properties of One-vs-One

- Many more classifiers than OvR.
- Each classifier is trained on only two classes.
- Historically used in SVM implementations (e.g. libsvm).
- Computational cost grows quickly with number of classes.

Originally, Logistic Regression was binary only.

It could be extended using OvR.

Modern implementations instead use a direct multi-class formulation:

Multinomial Logistic Regression (Softmax)

$$P(y = k|x) = \frac{e^{w_k^T x}}{\sum_j e^{w_j^T x}}$$

This is called the **softmax function**.

We will see this again when we study Neural Networks.

- OvR and OvO:
 - Reduce problem to multiple binary classifiers
- Softmax:
 - Single model
 - Jointly models all classes

Modern LogisticRegression uses softmax.

Modern SVM typically uses OvR.

All three strategies are still important concepts.